چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۵

کار پس‌زمینه — بخش ۱: WorkManager، حالت Doze و App Standby



اسماعیل مرادی

@esmaeeil-moradi700

چرا کار پس‌زمینه در ۲۰۲۶ مهم است؟

کار پس‌زمینه جایی است که اپ‌ها اعتماد کاربر را از دست می‌دهند: باتری‌ای که خالی می‌شود، syncای که هیچ‌وقت انجام نشد، crash بعد از بالا بردن targetSdk.
مصاحبه‌گر با این موضوع می‌سنجد که می‌دانید **سیستم به شما اجازهٔ چه کاری را می‌دهد — و چه کاری را نه**:

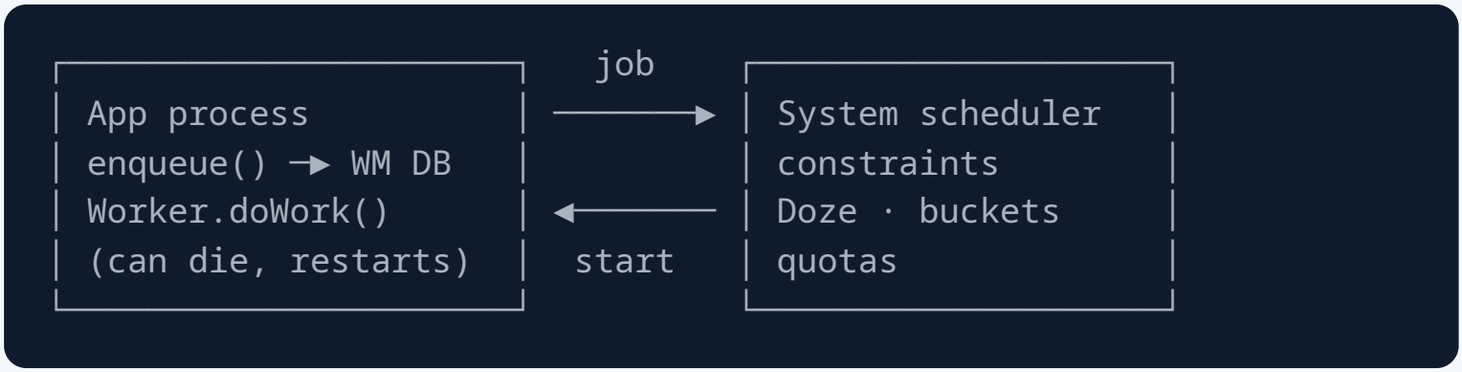
- کاری که باید از مرگ process جان سالم به در ببرد مال **WorkManager** است، که روی JobScheduler ساخته شده
- **constraintها، retry و unique work** به‌جای زمان‌بندهای دست‌ساز
- **expedited work** برای کارهای فوری و کوتاه — که هزینه‌اش را با quota می‌پردازید
- **Doze و App Standby bucketها** تعیین می‌کنند کار شما *واقعاً کِی* اجرا شود
- قواعد تقریباً با هر نسخه عوض می‌شوند — بدانید **کدام نسخه** هر محدودیت را آورده

سطح	سؤال‌ها
جونیور	WorkManager چیست، اولین worker، constraintها و کار دوره‌ای
مید	زنجیره‌کردن، unique work، retry و backoff، مشاهده و تست
سینیور	کار expedited و طولانی، Doze و App Standby، محدودیت‌ها به تفکیک نسخه
آرکیتکت	لایهٔ کار پس‌زمینه، sync کم‌مصرف، observability



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: process شما زمان‌بند نیست



- WorkManager درخواست را **ذخیرهٔ پایدار** می‌کند، بعد آن را به زمان‌بند سیستم (JobScheduler) می‌سپارد
- process شما هر لحظه ممکن است بمیرد؛ **سیستم** job را به خاطر می‌سپارد و برای اجرایش process را دوباره بالا می‌آورد
- اینکه کِی اجرا شود را constraintها، Doze، standby bucket و quotaها تعیین می‌کنند — **نه شما**

سؤال واحد پشت همهٔ جواب‌های امروز

«اگر process همین حالا بمیرد، سر این کار چه می‌آید؟» اگر جواب «گم می‌شود» است و این قابل‌قبول نیست، جای این کار در یک coroutine نیست.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

WorkManager چیست، اولین worker، constraintها و کار دوره‌ای



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. WorkManager چیست — و چه نیست

WorkManager، API جت‌پک برای **کار پایدار** (persistent work) است: کاری که حتی اگر کاربر از اپ خارج شود، process کشته شود یا دستگاه ری‌استارت شود، باید انجام شود.

- درخواست‌ها در دیتابیس خود WorkManager ذخیره می‌شوند
- روی API 23 و بالاتر از طریق **JobScheduler** زمان‌بندی می‌کند؛ تا وقتی process زنده است، می‌تواند کار آماده را فوراً هم شروع کند
- بعد از ری‌استارت دستگاه کارهایش را دوباره زمان‌بندی می‌کند — به boot receiver خودتان نیازی نیست

نیاز	ابزار
کار وابسته به یک صفحه (بارگذاری داده، ذخیره با کلیک)	coroutine در `viewModelScope`
باید تمام شود حتی اگر اپ بسته شود؛ می‌تواند منتظر شرایط بماند	**WorkManager**
قابل‌مشاهده برای کاربر و مداوم (پخش، مسیریابی، تماس)	foreground service — روز ۱۶
باید سر یک ساعت دقیق اجرا شود	exact alarm — روز ۱۶



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*WorkManager روی دستگاه‌های قدیمی از Firebase JobDispatcher یا GCMNetworkManager استفاده می‌کند.*» — قدیمی است (نسخهٔ پارسال هم همین را گفته بود). Firebase JobDispatcher سال‌ها پیش deprecated شد. امروز: روی API 23 و بالاتر JobScheduler، و فقط پایین‌تر از آن زمان‌بندی مبتنی بر AlarmManager.

تلهٔ مصاحبه

«*تضمین‌شده یعنی همین الان اجرا می‌شود — و همیشه.*» — یعنی **بالاخره، وقتی constraintها اجازه دهند**. اگر کاربر اپ را **force stop** کند، سیستم همهٔ jobها و alarmهایش را لغو می‌کند؛ تا اپ دوباره باز نشود هیچ چیز اجرا نمی‌شود (بعد WorkManager دوباره زمان‌بندی می‌کند). بعضی نسخه‌های سازندگان (OEM) از این هم سخت‌گیرترند.

سؤال تکمیلی

«*چرا یک coroutine در scope سطح اپلیکیشن راه نیندازیم؟*» — با process می‌میرد، constraint نمی‌فهمد و هیچ‌وقت retry نمی‌شود. برای کاری که از دست رفتنش مهم نیست (گرم کردن cache) خوب است — نه برای «حتماً باید انجام شود».



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. اولین worker شما

```
class SyncWorker(
    context: Context,
    params: WorkerParameters,
    private val repo: SyncRepository, // WorkerFactory, Q11
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val userId = inputData.getString(KEY_USER_ID)
            ?: return Result.failure()
        return try {
            repo.sync(userId)
            Result.success()
        } catch (e: IOException) {
            Result.retry()
        }
    }
}
```

- `Result.success()` یعنی کار تمام شد؛ `Result.retry()` یعنی بعداً با backoff دوباره اجرا شود؛ `Result.failure()` یعنی متوقف شو — کارهای وابسته هم fail می‌شوند
- برای Kotlin از `CoroutineWorker` استفاده کنید؛ `Worker` کد blocking را روی یک executor پس‌زمینه اجرا می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. enqueue کنید — و محدودیت‌هایش را بدانید

```
val request = OneTimeWorkRequestBuilder<SyncWorker>()
    .setInputData(workDataOf(KEY_USER_ID to userId))
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()
    )
    .build()

WorkManager.getInstance(context).enqueue(request)
```

تلهٔ مصاحبه

«`CoroutineWorker` روی `Dispatchers.IO` *اجرا می‌شود.*» — روی `Dispatchers.Default` اجرا می‌شود. APIهای `suspend` که main-safe هستند (Room، Retrofit، Ktor) مشکلی ندارند؛ فراخوانی‌های blocking (streamهای فایل، SDKهای قدیمی) را در `withContext(Dispatchers.IO)` بپیچید.

تلهٔ مصاحبه

worker نامحدود نیست: هر اجرا **حداکثر ۱۰ دقیقه** وقت دارد، بعد متوقف می‌شود (coroutine لغو می‌شود) و دوباره زمان‌بندی می‌شود. کار طولانی‌تر راه دیگری می‌خواهد — سؤال ۸.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. constraintها: فقط وقتی منطقی است اجرا کن

شرط	فراخوانی Builder
شبکه	`setRequiredNetworkType(NetworkType.CONNECTED)` — و همین‌طور `UNMETERED`، `NOT_ROAMING`، `METERED`
در حال شارژ	`setRequiresCharging(true)`
باتری کم نباشد	`setRequiresBatteryNotLow(true)`
حافظه کم نباشد	`setRequiresStorageNotLow(true)`
دستگاه بیکار باشد	`setRequiresDeviceIdle(true)` — می‌تواند کار را ساعت‌ها عقب بیندازد
تغییر محتوا	`addContentUriTrigger(uri, true)` — مثلاً عکس‌های جدید

- اگر یک constraint **وسط اجرای worker** برقرار نماند، WorkManager آن را متوقف می‌کند و بعداً دوباره اجرا می‌کند
- constraintها با هم AND می‌شوند: همه باید برقرار باشند

تلهٔ مصاحبه

نسخهٔ پارسال constraintهایی مثل `DeviceIdle.ON` و `Charging.ON` را فهرست کرده بود. **چنین چیزهایی وجود ندارند.** API واقعی همان setterهای `Constraints.Builder` در جدول بالاست — اسم API ساختگی یعنی علامت خطر فوری.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. کار دوره‌ای

```
// Every 6 h, somewhere in the last 1 h of each period
val refresh = PeriodicWorkRequestBuilder<RefreshWorker>(
    6, TimeUnit.HOURS,
    1, TimeUnit.HOURS, // flex window
).setConstraints(unmeteredAndNotLowBattery)
    .build()
```

- حداقل بازه: **۱۵ دقیقه**؛ حداقل flex: **۵ دقیقه**
- کار دوره‌ای زنجیره نمی‌شود، و **هیچ‌وقت** `SUCCEEDED` **نمی‌شود** — بعد از هر اجرا به `ENQUEUED` برمی‌گردد

تلهٔ مصاحبه

«*بازهٔ ۱۵ دقیقه یعنی هر ۱۵ دقیقه اجرا می‌شود.*» — این یک **حداقل** است. Doze، standby bucket و constraintها اجرا را عقب می‌اندازند. زمان دقیق لازم دارید؟ alarm. دادهٔ لحظه‌ای لازم دارید؟ push (روز ۱۶).



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

زنجیره‌کردن، unique work، retry و backoff، مشاهده و تست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. زنجیره‌کردن کارها

```
WorkManager.getInstance(context)
    .beginUniqueWork(
        "upload-$albumId",
        ExistingWorkPolicy.KEEP,
        listOf(compressA, compressB), // run in parallel
    )
    .then(upload)   // starts when both succeeded
    .then(notify)
    .enqueue()
```

- لیستی که به `beginWith()` یا `beginUniqueWork()` یا `then()` بدهید **موازی** اجرا می‌شود
- `then()` صبر می‌کند تا **همهٔ** والدها موفق شوند
- `outputData` والد، `inputData` فرزند می‌شود. اگر چند والد باشد، یک `InputMerger` آن‌ها را ترکیب می‌کند: `OverwritingInputMerger` (پیش‌فرض) یا `ArrayCreatingInputMerger`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. جاهایی که زنجیره‌ها غافلگیرتان می‌کنند

تلهٔ مصاحبه

«با `parallel()` *workerها را کنار هم اجرا کن.*» — در WorkManager **هیچ** `parallel()` ای وجود ندارد (نسخهٔ پارسال از آن استفاده کرده بود). موازی‌سازی یعنی دادن یک لیست به `beginWith()` یا `then()`، یا `WorkContinuation.combine()`.

تلهٔ مصاحبه

«*پاک‌سازی را آخر زنجیره بگذار تا همیشه اجرا شود.*» — اگر هر مرحله‌ای fail یا لغو شود، **همهٔ مراحل وابسته** هم `FAILED` یا `CANCELLED` می‌شوند. پاک‌سازی‌ای که حتماً باید انجام شود، جایش در بلوک `finally` خود worker است، یا در یک کار جداگانه.

تلهٔ مصاحبه

`Data` حداکثر **۱۰ کیلوبایت** است (`Data.MAX_DATA_BYTES`)؛ ساختن نمونهٔ بزرگ‌تر exception می‌دهد. ID و URI بفرستید — هرگز bitmap یا JSON حجیم. خود داده در Room یا یک فایل می‌ماند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. unique work: دقیقاً یک نمونه از هر نوع

ExistingWorkPolicy	وقتی کاری با همین اسم از قبل وجود دارد
KEEP	درخواست جدید نادیده گرفته می‌شود
REPLACE	کار موجود لغو می‌شود (حتی در حال اجرا) و کار جدید enqueue می‌شود
APPEND	کار جدید بعد از آن اجرا می‌شود؛ اگر کار موجود fail شود، کار جدید هم fail می‌شود
APPEND_OR_REPLACE	مثل APPEND، اما اگر کار قبلی fail یا لغو شده باشد، زنجیرهٔ تازه‌ای شروع می‌کند

ExistingPeriodicWorkPolicy	اثر
KEEP	زمان‌بندی موجود حفظ می‌شود
UPDATE	مشخصات در جا به‌روز می‌شود؛ worker در حال اجرا قطع نمی‌شود
CANCEL_AND_REENQUEUE	لغو و شروع از اول (جایگزین REPLACE که deprecated شده)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. enqueue هنگام شروع اپ — به شکل امن

```
// Safe to call on every app start
WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "daily-refresh",
    ExistingPeriodicWorkPolicy.UPDATE,
    refresh,
)
```

- **اسم یکتا** (unique name) یعنی حداکثر یک نمونه. **تگ** (tag) برچسبی روی چند درخواست است، برای مشاهده یا لغو گروهی آن‌ها (`cancelAllWorkByTag`)

تلهٔ مصاحبه

`CANCEL_AND_REENQUEUE` (یا `enqueue()` ساده) در `Application.onCreate()` با هر بار باز شدن اپ زمان‌بندی را از نو شروع می‌کند — یا نسخه‌های تکراری روی هم می‌سازد — و ممکن است sync در حال اجرا را لغو کند. از `KEEP` یا `UPDATE` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. retry و backoff

```
override suspend fun doWork(): Result {
    if (runAttemptCount >= MAX_ATTEMPTS) return Result.failure()
    return try {
        repo.pushPendingChanges()
        Result.success()
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Result.retry()
    } catch (e: HttpException) {
        if (e.code() >= 500) Result.retry()
        else Result.failure()
    }
}
```

```
.setBackoffCriteria(
    BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS,
)
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. یک سیاست retry خوب چه می‌داند

- پیش‌فرض: **نمایی، شروع از ۳۰ ثانیه** (۳۰ ثانیه، ۱ دقیقه، ۲ دقیقه...)، بازهٔ مجاز **۱۰ ثانیه تا ۵ ساعت**؛ حالت `LINEAR` هم هست
- تأخیر backoff یک **حداقل** است — constraintها و Doze همچنان اعمال می‌شوند
- با `runAttemptCount` می‌توانید بعد از N تلاش دست بکشید

تلهٔ مصاحبه

`runCatching { }` داخل `doWork()`، `CancellationException` را هم می‌گیرد. وقتی سیستم worker را متوقف می‌کند (از دست رفتن constraint، سقف ۱۰ دقیقه)، worker به‌جای توقف همکارانه به کارش ادامه می‌دهد. آن را دوباره throw کنید، یا فقط exceptionهای مشخص را بگیرید.

تلهٔ مصاحبه

retry کردن همه چیز. خطای `400` یا `401` در هر تلاش دقیقاً همان‌طور fail می‌شود — `failure()` برگردانید (یا اول احراز هویت را درست کنید). فقط خطاهای **گذرا** را retry کنید: I/O، خطاهای 5xx و 429.

RETRY یعنی تکرار

یک retry ممکن است درخواستی را دوباره بفرستد که سرور قبلاً پردازش کرده. با هر عملیات نوشتن یک **idempotency key** بفرستید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. مشاهدهٔ کارها از UI

```
// ViewModel
val upload: StateFlow<UploadUi> = workManager
    .getWorkInfosForUniqueWorkFlow("upload-$albumId")
    .map { infos -> infos.firstOrNull().toUploadUi() }
    .stateIn(
        viewModelScope,
        SharingStarted.WhileSubscribed(5_000),
        UploadUi.Idle,
    )

// Worker
setProgress(workDataOf(KEY_PROGRESS to percent))
```

- وضعیت‌ها: `ENQUEUED` سپس `RUNNING` و در پایان `SUCCEEDED` یا `FAILED` یا `CANCELLED`؛ وضعیت `BLOCKED` یعنی منتظر والدهاست
- `progress` فقط در حالت `RUNNING` در دسترس است؛ `outputData` فقط بعد از پایان کار

تلهٔ مصاحبه

`WorkInfo` را منبع داده فرض کردن. وضعیت را از آن نشان دهید؛ نتیجه از repository (یعنی Room) می‌آید — یک منبع حقیقت (روز ۳). ضمناً worker دوره‌ای هیچ‌وقت به `SUCCEEDED` نمی‌رسد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. تست workerها

```
@Test fun retriesOnNetworkError() = runTest {
    val worker = TestListenableWorkerBuilder<SyncWorker>(ctx)
        .setInputData(workDataOf(KEY_USER_ID to "42"))
        .setWorkerFactory(FakeWorkerFactory(OfflineRepo()))
        .build()

    val result = worker.doWork()

    assertEquals(ListenableWorker.Result.retry(), result)
}
```

- کتابخانهٔ `work-testing`: کلاس `TestListenableWorkerBuilder` متد `doWork()` را مستقیم اجرا می‌کند — با input، `setRunAttemptCount()` و یک factory جعلی
- برای درخواست‌ها، زنجیره‌ها و constraintها: `WorkManagerTestInitHelper` همراه با `TestDriver` (`setAllConstraintsMet(id)`، `setInitialDelayMet(id)`، `setPeriodDelayMet(id)`)

تلهٔ مصاحبه

منتظر ماندن برای WorkManager واقعی با `Thread.sleep()`. WorkManager تست را با `SynchronousExecutor` مقداردهی کنید و زمان و constraintها را با `TestDriver` جلو ببرید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

کار expedited و طولانی، Doze و App Standby، محدودیت‌ها به تفکیک نسخه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. کار expedited

```
val send = OneTimeWorkRequestBuilder<SendMessageWorker>()
    .setExpedited(
        OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST
    )
    .build()

class SendMessageWorker(
    ctx: Context,
    params: WorkerParameters,
) : CoroutineWorker(ctx, params) {
    // Below Android 12 it runs as a foreground service
    override suspend fun getForegroundInfo() =
        ForegroundInfo(NOTIFICATION_ID, sendingNotification())

    override suspend fun doWork(): Result = send()
}
```

- **در اولین فرصت** شروع می‌شود، حتی در پس‌زمینه، و کمتر تحت تأثیر تعویق‌های صرفه‌جویی باتری است
- در اندروید ۱۲ و بالاتر به شکل یک **expedited job** اجرا می‌شود که با یک **quota** زمانی برای هر اپ محدود است؛ اگر quota تمام شود، یا به شکل کار عادی اجرا می‌شود یا دور ریخته می‌شود
- نمی‌تواند دوره‌ای باشد و initial delay هم نمی‌پذیرد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. expedited در برابر طولانی‌مدت

تلهٔ مصاحبه

«*هر چیز مهمی را expedited کن.*» — quota مشترک است. اگر آن را خرج analytics کنید، تأیید پرداخت دیر اجرا می‌شود. expedited یعنی کارهای کوتاهی که کاربر منتظرشان است: فرستادن پیام، تأیید پرداخت.

تلهٔ مصاحبه

override نکردن `getForegroundInfo()` باعث می‌شود کار expedited **روی دستگاه‌های پایین‌تر از اندروید ۱۲ crash کند**؛ چون آنجا WorkManager برای نمایش notification پیش‌زمینه به آن نیاز دارد.

بیشتر از ۱۰ دقیقه؟ worker طولانی‌مدت `setForeground()` را صدا می‌زند و به یک foreground service تبدیل می‌شود:

```
setForeground(
    ForegroundInfo(
        NOTIFICATION_ID, progressNotification(),
        ServiceInfo.FOREGROUND_SERVICE_TYPE_DATA_SYNC,
    )
)
```

از همهٔ قواعد FGS پیروی می‌کند — نوع سرویس باید روی `SystemForegroundService` مربوط به WorkManager در manifest شما اعلام شود (اندروید ۱۴)، محدودیت شروع از پس‌زمینه (اندروید ۱۲)، و سقف زمانی `dataSync` (اندروید ۱۵). جزئیات: روز ۱۶.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. Doze: وقتی گوشی می‌خوابد چه چیزهایی متوقف می‌شود

Doze (اندروید ۶٫۰) وقتی شروع می‌شود که دستگاه از شارژ جدا، ثابت و با صفحهٔ خاموش باشد؛ نسخهٔ سبک‌ترش (اندروید ۷٫۰) در حال حرکت هم اعمال می‌شود. کارها در **پنجره‌های نگهداری** (maintenance window) کوتاه دسته‌بندی می‌شوند.

در Doze	رفتار
دسترسی به شبکه	بیرون از پنجره‌های نگهداری قطع است
wake lockها	نادیده گرفته می‌شوند
jobها و WorkManager	به پنجرهٔ نگهداری بعدی موکول می‌شوند
alarmهای معمولی	عقب می‌افتند
`setAndAllowWhileIdle()` و `setExactAndAllowWhileIdle()`	اجرا می‌شوند، اما برای هر اپ محدودیت نرخ دارند
پیام FCM با اولویت بالا	تحویل داده می‌شود؛ اپ یک پنجرهٔ کوتاه در allowlist می‌گیرد

```
adb shell dumpsys battery unplug
adb shell dumpsys deviceidle force-idle   # enter Doze
adb shell dumpsys deviceidle unforce
adb shell dumpsys battery reset
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. App Standby bucketها (اندروید ۹ به بعد)

سیستم هر اپ را بر اساس اینکه اخیراً و چقدر استفاده شده، در یک bucket می‌گذارد:

bucket	اپ نمونه	کار پس‌زمینه
Active	همین حالا در حال استفاده	بدون محدودیت
Working set	مرتب استفاده می‌شود	jobها و alarmها کمی عقب می‌افتند
Frequent	زیاد استفاده می‌شود، نه هر روز	بیشتر عقب می‌افتند
Rare	به‌ندرت استفاده می‌شود	بیشترین تعویق
Restricted (اندروید ۱۲ به بعد)	مدت‌ها استفاده نشده، یا بسیار پرمصرف	تقریباً روزی یک بار

```
adb shell am set-standby-bucket com.example.app rare
adb shell am get-standby-bucket com.example.app
```

- در کد: `UsageStatsManager.appStandbyBucket` — کنار متریک‌های sync لاگش کنید

تلهٔ مصاحبه

نسخهٔ پارسال «Exempted» را هم یک bucket حساب کرده بود؛ آن bucket داخلی سیستم است و اپ‌ها همان پنج مورد بالا را می‌بینند. راه‌حل تعویق هم این **نیست** که از همه بخواهید battery optimization را خاموش کنند: `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` طبق سیاست Play محدود است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. محدودیت‌های پس‌زمینه، نسخه به نسخه

اندروید	API	چه چیزی برای کار پس‌زمینه عوض شد
۶٫۰	23	حالت Doze و App Standby
۷٫۰	24	Doze در حال حرکت؛ receiverهای manifest دیگر `CONNECTIVITY_ACTION` نمی‌گیرند
۸٫۰	26	محدودیت سرویس‌های پس‌زمینه؛ محدودیت broadcastهای implicit
۹	28	App Standby bucketها
۱۲	31	bucket محدودشده؛ expedited job؛ محدودیت شروع FGS؛ مجوز exact alarm
۱۴	34	اجباری شدن نوع FGS؛ jobهای انتقال داده به درخواست کاربر
۱۵	35	سقف زمانی FGS از نوع `dataSync`؛ محدودیت FGS بعد از boot
۱۶	36	quota زمان اجرای jobها به standby bucket و وضعیت اپ بستگی دارد

تلهٔ مصاحبه

«*اگر کار سریع است، `startService()` از یک receiver مشکلی ندارد.*» — از اندروید ۸٫۰، اپ پس‌زمینه `IllegalStateException` می‌گیرد. کار را به WorkManager بسپارید.

با نسخه جواب بدهید

«این قاعدهٔ اندروید ۱۲ برای targetSdk ۳۱ به بالاست» خیلی بهتر از «اندروید اجازه نمی‌دهد» است. بیشتر محدودیت‌ها تابع **targetSdk** هستند؛ Doze و bucketها روی همهٔ اپ‌ها اعمال می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

لایهٔ کار پس‌زمینه، sync کم‌مصرف، observability



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. طراحی لایهٔ کار پس‌زمینه

```
// domain — no WorkManager imports
interface SyncScheduler {
    fun requestSync()
}

// data — the only place that knows WorkManager
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted context: Context,
    @Assisted params: WorkerParameters,
    private val repo: SyncRepository,
) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result =
        repo.drainPendingChanges().toWorkResult()
}
```

- workerها **نازک** هستند: input را می‌خوانند، یک repository را صدا می‌زنند و نتیجه را به یک `Result` تبدیل می‌کنند
- کار **idempotent** است — ممکن است دو بار اجرا شود. ID بفرستید؛ state در Room می‌ماند
- در Hilt: کلاس `Application` رابط `Configuration.Provider` را با `HiltWorkerFactory` پیاده می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. تله‌هایی که در مقیاس بزرگ گیرتان می‌اندازند

تلهٔ مصاحبه

یک `WorkerFactory` سفارشی دارید اما initializer پیش‌فرض هنوز در manifest است؛ در نتیجه WorkManager با factory پیش‌فرض شروع می‌شود و workerهای injectشده ساخته نمی‌شوند.
`WorkManagerInitializer` را از `InitializationProvider` حذف کنید.

تلهٔ مصاحبه

تغییر نام یا جابه‌جایی کلاس یک worker. درخواست‌های enqueueشده **اسم کلاس** را ذخیره کرده‌اند؛ بعد از آپدیت، WorkManager نمی‌تواند آن را بسازد و کار fail می‌شود. کلاس قدیمی را برای یک نسخه به‌عنوان واسطه نگه دارید، یا هنگام ارتقا کار را با اسم یکتا لغو و دوباره enqueue کنید.

تلهٔ مصاحبه

`KEEP` درخواست sync را که **وسط اجرای sync قبلی** می‌رسد بی‌صدا دور می‌ریزد — تغییراتی که وسط اجرا ایجاد شده‌اند تا trigger بعدی منتظر می‌مانند. بگذارید worker صف تغییرات معلق را از Room آن‌قدر خالی کند تا چیزی باقی نماند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. استراتژی sync کم‌مصرف

الگو	چرا
push برای sync: پیام FCM، سپس یک کار یک‌باره	فقط وقتی چیزی عوض شده sync می‌کند
تور ایمنی دوره‌ای: ساعتی، نه دقیقه‌ای	pushهای گم‌شده را جبران می‌کند؛ کار سنگین روی شبکهٔ بدون محدودیت و در حال شارژ
ادغام درخواست‌ها با unique work	یک بار بیدار شدن به‌جای ده بار
sync هنگام باز شدن اپ	کاربر اینجاست و دستگاه بیدار است
بازه‌های تنظیم‌شده از سرور (Remote Config)	تغییر ریتم بدون انتشار نسخهٔ جدید

```
val nightly = PeriodicWorkRequestBuilder<BackupWorker>(
    24, TimeUnit.HOURS,
).setInitialDelay(Random.nextLong(0, 60), TimeUnit.MINUTES)
    .setConstraints(unmeteredAndCharging)
    .build()
```

تلهٔ مصاحبه

همهٔ کلاینت‌ها ساعت ۰۰:۰۰ sync می‌کنند — یک حملهٔ DDoS خودساخته روی بک‌اند. jitter اضافه کنید، بگذارید WorkManager اجراها را پخش کند و به `Retry-After` احترام بگذارید. و poll هر ۱۵ دقیقه هیچ‌وقت «لحظه‌ای» نیست: از push استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. observability: دیباگ کاری که نمی‌بینید

سؤال	ابزار
کار من enqueue شده، block شده یا در حال اجراست؟	ابزار **Background Task Inspector** در Android Studio
چرا اجرا نمی‌شود؟	`adb shell dumpsys jobscheduler`
نمای خود WorkManager	broadcast تشخیصی پایین — در buildهای debug، خروجی در Logcat
همین حالا یک job را اجرا کن	`adb shell cmd jobscheduler run -f <pkg> <jobId>`
رفتار وقتی اپ کم استفاده می‌شود	`am set-standby-bucket` و `deviceidle force-idle` (سؤال ۹)
اثر روی باتری در دنیای واقعی	گزارش Android vitals دربارهٔ بیدارباش‌های بیش از حد و wakelockها

```
adb shell am broadcast -p com.example.app \
    -a androidx.work.diagnostics.REQUEST_DIAGNOSTICS
```

تلهٔ مصاحبه

«*روی گوشی من کار می‌کند.*» — گوشی شما در حال شارژ است، روی Wi-Fi، در bucket فعال. کاربران واقعی در Rare هستند، روی اینترنت حجمی، در Doze. این حالت‌ها را عمداً تست کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. متریک‌های production از هر worker

```
abstract class TrackedWorker(
    ctx: Context, params: WorkerParameters,
    private val metrics: WorkMetrics,
) : CoroutineWorker(ctx, params) {

    protected abstract suspend fun work(): Result

    final override suspend fun doWork(): Result {
        val name = this::class.simpleName.orEmpty()
        return try {
            val result = work()
            metrics.finished(name, runAttemptCount, result)
            result
        } catch (e: CancellationException) {
            metrics.stopped(name, runAttemptCount)
            throw e
        }
    }
}
```

- نرخ موفقیت، شکست و توقف، تعداد تلاش به ازای هر موفقیت، و **فاصلهٔ enqueue تا شروع** را ثبت کنید — به تفکیک standby bucket و نسخهٔ اندروید

جواب سینیور در یک جمله

«WorkManager تضمین می‌کند کار *به خاطر* سپرده شود، نه اینکه کِی اجرا شود. من برای اجرای دیرهنگام، تکراری و متوقف‌شده طراحی می‌کنم — workerهای idempotent، retry محدود، و متریک‌هایی که نشان می‌دهند چقدر دیر.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۵ — خلاصهٔ یک‌صفحه‌ای

- WorkManager یعنی کار **پایدار** که از مرگ process و ری‌استارت عبور می‌کند؛ روی JobScheduler.
- force stop همه چیز را تا باز شدن دوبارهٔ اپ لغو می‌کند — «تضمین‌شده» یعنی *بالاخره*.
- `CoroutineWorker` روی `Dispatchers.Default` اجرا می‌شود؛ هر اجرا حداکثر **۱۰ دقیقه**.
- constraintها setterهای `Constraints.Builder` هستند؛ کار دوره‌ای حداقل **۱۵ دقیقه** است و `SUCCEEDED` نمی‌شود.
- در زنجیره، لیست‌ها موازی‌اند و یک شکست همهٔ وابسته‌ها را fail می‌کند. `Data` حداکثر ۱۰ کیلوبایت.
- unique work: هنگام شروع اپ `KEEP` یا `UPDATE`، هرگز enqueue کورکورانه.
- فقط خطای گذرا را retry کنید؛ `CancellationException` را rethrow کنید؛ idempotency key بفرستید.
- expedited یعنی فوری و کوتاه، با quota؛ زیر اندروید ۱۲ به `getForegroundInfo()` نیاز دارد.
- Doze و standby bucketها تعیین می‌کنند کار **کِی** اجرا شود؛ با `adb` تست کنید.
- آرکیتکت: workerهای نازک و idempotent پشت interface، push برای sync به‌علاوهٔ تور ایمنی دوره‌ای، متریک برای هر worker.

روز ۱۶ در راه است: کار پس‌زمینه، بخش ۲ — سرویس‌ها، alarmها و push. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700